

Solution Architecture

Date	18 September 2026
Team ID	SWTID-2026-3342
Project Name	AI BlogNest API
Maximum Marks	5 Marks

Solution Architecture Diagram

AI BlogNest follows a layered REST architecture: an API client sends requests through the Express gateway, which routes them (after JWT and role verification on private routes) to Controllers, which delegate business logic to Models or, for generative features, to the AI Integration Layer.

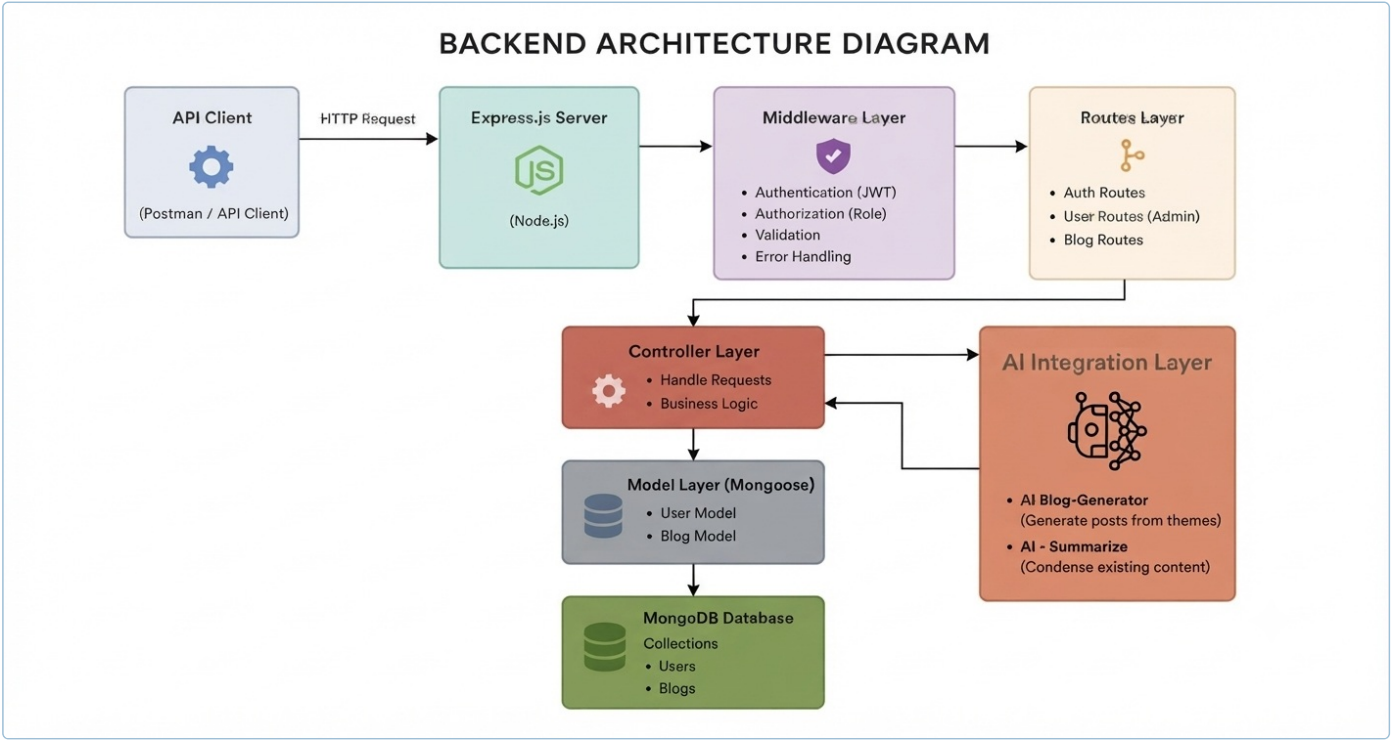
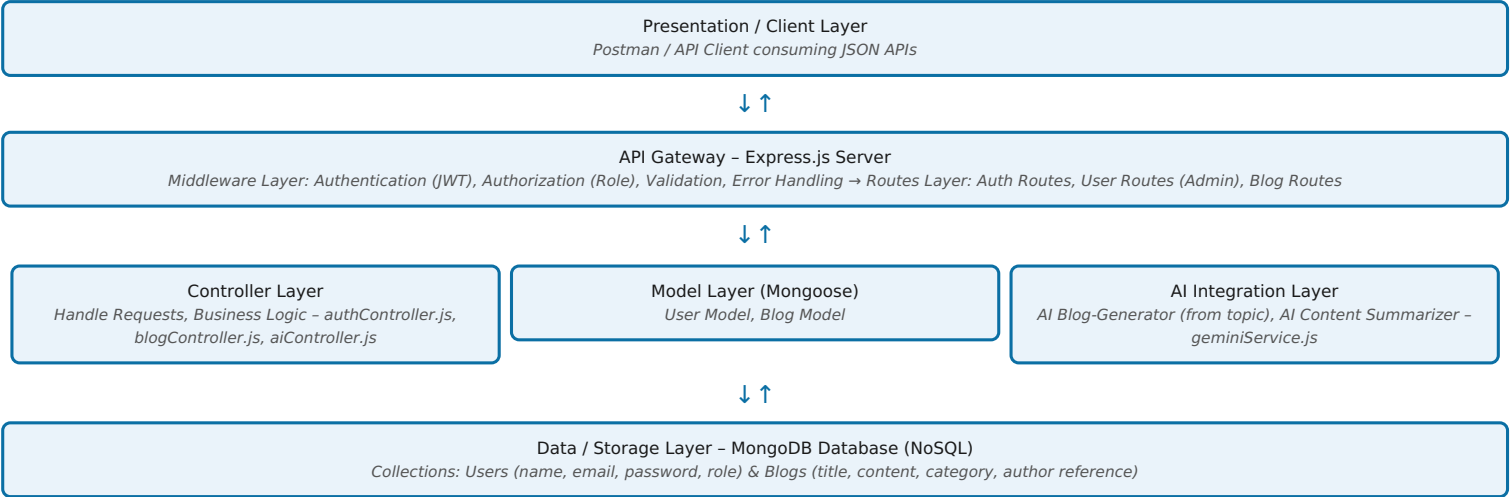


Fig. 1 – AI BlogNest backend architecture as implemented (API Client → Express Server → Middleware Layer → Routes Layer → Controller Layer ↔ AI Integration Layer → Model Layer → MongoDB Database).

Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	REST client used to send and inspect JSON requests/responses during development and grading	Postman, Thunder Client
API Gateway	Central Express.js entry point (app.js); applies cors , JSON parsing, morgan logging, and dispatches to /api/auth , /api/blogs and /api/ai route groups, then a shared error middleware	Express.js, cors, morgan, errorMiddleware
Microservices (Controllers / Services)	Encapsulate domain logic: authController (register/login), blogController (CRUD, drafts, categories, tags, comments), aiController + geminiService (generate-blog, summarize)	Node.js, Express Router, Google Gemini AI SDK
Database	Persists Users, Blogs and Comments with Mongoose-enforced schema validation and one-to-many relations	MongoDB, Mongoose

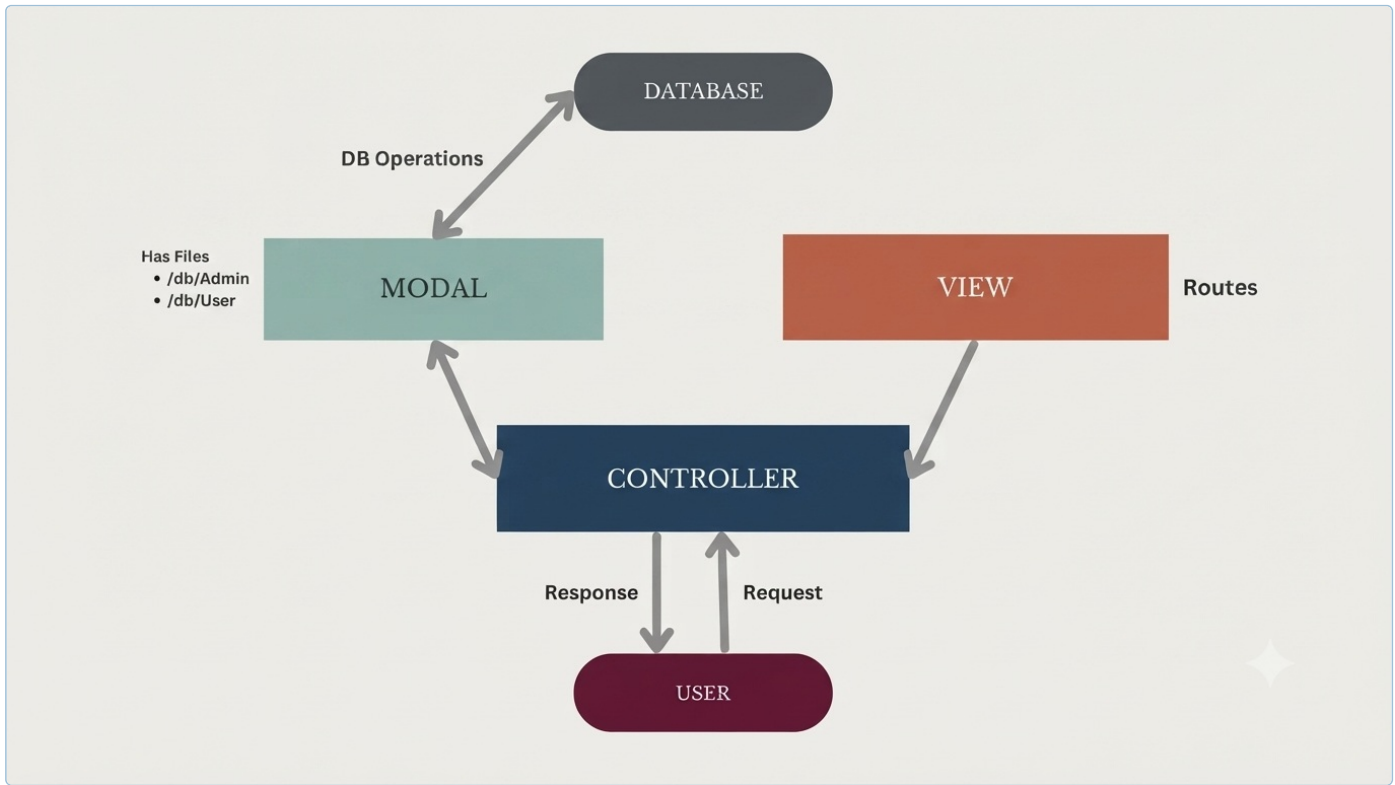


Fig. 2 - MVC pattern used within AI BlogNest: Modal/Model (Mongoose schemas & DB operations), Controller (request/response handling), View (routes - there is no template UI since this is a headless backend).